Université d'Ottawa
Faculté de génie

École de science
d'informatique
et de génie électrique

uOttawa
L'Université canadienne
Canada's university

University of Ottawa
Faculty of Engineering

School of Electrical
Engineering
and Computer Science

# Devoir 2
# CSI2520 Paradigmes de Programmation
**Hiver 2026**

# *Assignment 2*
# CSI2120 Programming Paradigms
**Winter 2026**

## 6% - 18 points

**Question 1. (8 points)**

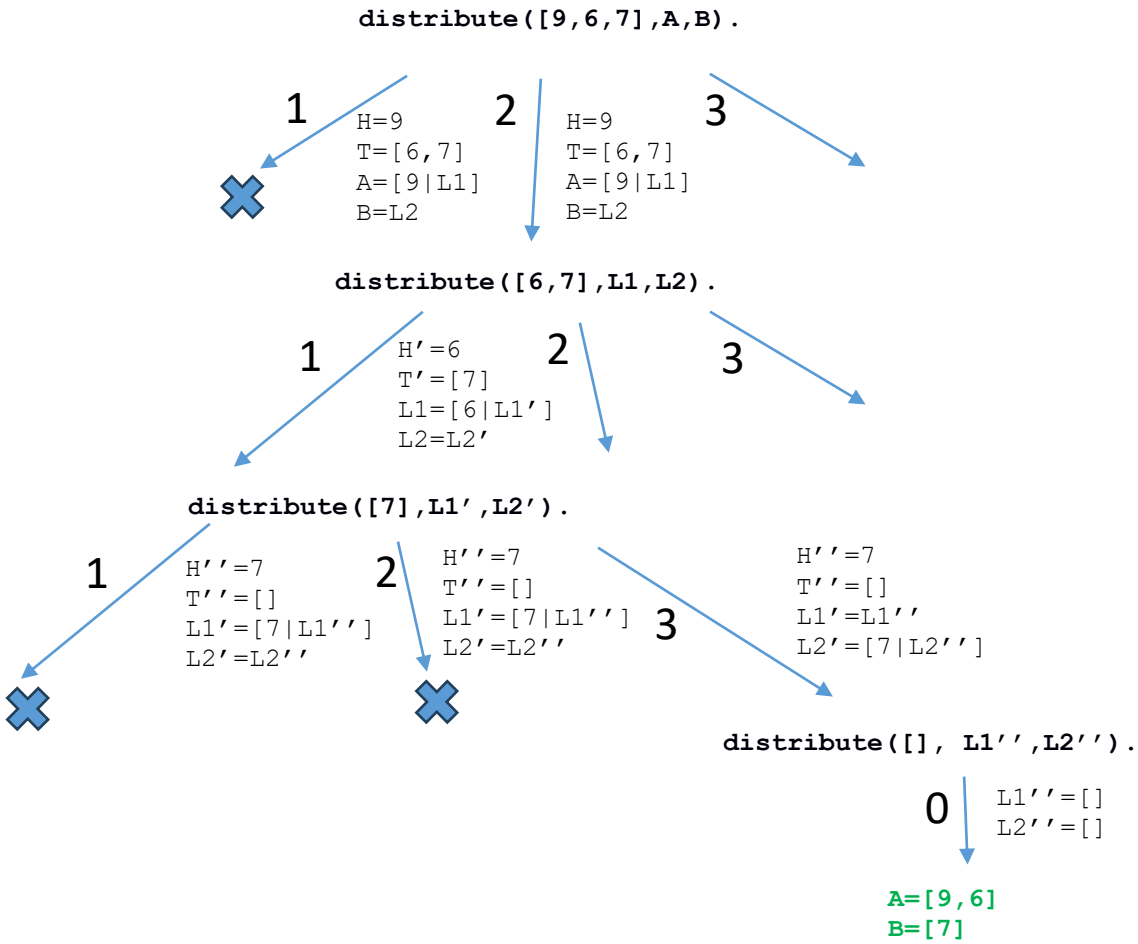
Soit le programme Prolog suivant :
*Consider the following Prolog program:*

```
divisible(X,N):- 0 is X mod N.        %0
distribute([], [], []).
distribute([H|T], [H|L1], L2) :-      %1
    divisible(H,2),
    distribute(T, L1, L2).
distribute([H|T], [H|L1], L2) :-      %2
    divisible(H,3),
    distribute(T, L1, L2).
distribute([H|T], L1, [H|L2]) :-      %3
    distribute(T, L1, L2).
```

a) [4 points] Dessiner l'arbre de résolution Prolog complet pour la requête ci-dessous. Bien montrer les solutions trouvées.
*Draw the complete Prolog search tree for the following query. Show all solutions found.*

```
?- distribute([9,6,7],A,B).
```

La page suivante montre l'arbre partiel menant à la première solution.
*The next page shows the partial tree for the first solution found.*


distribute([9,6,7],A,B).
1
H=9
T=[6,7]
A=[9|L1]
B=L2
2
H=9
T=[6,7]
A=[9|L1]
B=L2
3
distribute([6,7],L1,L2).
1
H'=6
T'=[7]
L1=[6|L1']
L2=L2'
2
3
distribute([7],L1',L2').
1
H''=7
T''=[]
L1'=[7|L1'']
L2'=L2''
2
H''=7
T''=[]
L1'=[7|L1'']
L2'=L2''
3
H''=7
T''=[]
L1'=L1''
L2'=[7|L2'']
distribute([], L1'',L2'').
0
L1''=[]
L2''=[]
A=[9,6]
B=[7]

b) [2+2 points] Pour chacune des coupes ajoutées ci-dessous, reproduire votre arbre complet et montrer quelles seront les branches élaguées par l'introduction de ces coupes (donner la liste des solutions trouvées)

*For each of the added cut below, reproduce your tree and show which branches will be pruned by these cuts (give the list of the solutions found)*

i)

```
divisible(X,N):- 0 is X mod N.        %0
distribute([], [], []).
distribute([H|T], [H|L1], L2) :-      %1
    divisible(H,2),!,
    distribute(T, L1, L2).
distribute([H|T], [H|L1], L2) :-      %2
    divisible(H,3),
    distribute(T, L1, L2).
distribute([H|T], L1, [H|L2]) :-      %3
    distribute(T, L1, L2).
```

ii)

```
divisible(X,N):- 0 is X mod N.        %0
distribute([], [], []).
distribute([H|T], [H|L1], L2) :-      %1
    divisible(H,2),!,
    distribute(T, L1, L2).
distribute([H|T], [H|L1], L2) :-      %2
    divisible(H,3),!,
    distribute(T, L1, L2).
distribute([H|T], L1, [H|L2]) :-      %3
    distribute(T, L1, L2).
```

**Question 2. (10 points)**

Soit la base de faits Prolog ci-dessous :
*Consider the following Prolog database:*

```
room(room101).
room(room202).
room(auditorium).
day(monday).
day(tuesday).
day(wednesday).
day(thursday).
day(friday).
work_hours(8,18). % events cannot start before, or end after
% event(event_name, day, start_hour, end_hour, room_name)
event(meeting1, monday, 9, 11, room101).
event(meeting2, monday, 14, 17, room101).
event(workshop, monday, 8, 17, room202).
event(cleanup, monday, 8, 18, auditorium).
event(conference, tuesday, 14, 16, room101).
event(board, tuesday, 9, 12, room202).
event(activity, tuesday, 10, 17, auditorium).
```

a) [2 points] Écrire le prédicat `busy_room(Day, Start, End, Room)` permettant de déterminer si une salle est occupée pour la période mentionnée.
*Write the predicate* `busy_room(Day, Start, End, Room)` *that determines if a room is occupied for the period specified.*

```
?- busy_room(monday, 13, 15, room101).
true .

?- busy_room(tuesday, 8, 11, R).
R = room202 ;
R = auditorium.

?- busy_room(D, 13, 15, room101).
D = monday ;
D = tuesday ;
false.
```

b) [2 points] Écrire le prédicat `free_room(Day, Start, End, Room)` permettant de déterminer si une salle est libre pour la période mentionnée.
*Write the predicate* `free_room(Day, Start, End, Room)` *that determines if a room is available for the period specified.*

```
?- free_room(D, 13, 15, room101).
D = wednesday ;
D = thursday ;
D = friday.

?- free_room(tuesday, 8, 11, R).
R = room101 ;
false.
```

c) [2 points] Écrire le prédicat `find_available_room(Day, Duration, Room, Start)` permettant de trouver une salle disponible pour un événement ayant une certaine durée.
*Write the predicate* `find_available_room(Day, Duration, Room, Start)` *that finds a room for an event of a specified duration.*

```
?- find_available_room(monday,2,Room,Start).
Room = room101,
Start = 11 ;
Room = room101,
Start = 12 ;
false.

?- find_available_room(Day,9,auditorium,Start).
Day = wednesday,
Start = 8 ;
Day = thursday,
Start = 8 ;
Day = friday,
Start = 8 ;
Day = wednesday,
Start = 9 ;
Day = thursday,
Start = 9 ;
Day = friday,
Start = 9 ;
false.
```

d) [2 points] Écrire le prédicat `hours_occupied(Room, Day, Hours)` permettant de calculer le nombre d'heures pendant laquelle une salle est occupée pour un jour donné.
*Write the predicate* `hours_occupied(Room, Day, Hours)` *that calculates the number of hours a room is occupied for a given day.*

```
?- hours_occupied(room101,J,H).
J = monday,
H = 5 ;
J = tuesday,
H = 2 ;
J = wednesday,
H = 0 ;
J = thursday,
H = 0 ;
J = friday,
H = 0.
```

e) [2 points] Écrire le prédicat `hours_occupied(Room, Hours)` permettant de calculer le nombre total d'heures qu'une salle est occupée dans la semaine.
*Write the predicate* `hours_occupied(Room, Hours)` *that calculates the total number of hours a room is occupied for the week.*

```
?- hours_occupied(room101,H).
H = 5.
```